

Processeurs ARM



Intro et Tutoriel

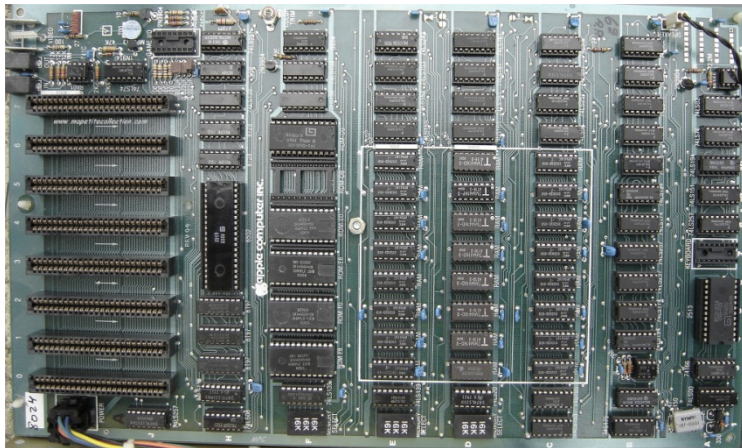
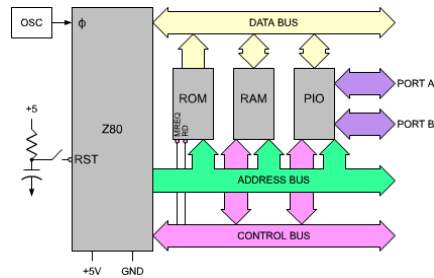
Rolf Ziegler, Microclub Lausanne, Janvier 2016

Agenda

- 40 ans de microcontrôleurs
- Architecture des processeurs
- Comparaison 8051, PIC, AVR, MSP430, ARM
- ARM Origine
- ARM Produits, Utilisation, Intégrateurs
- ARM Gamme des processeurs
- ARM Particularités
- ARM Outils
- Exemples pratiques

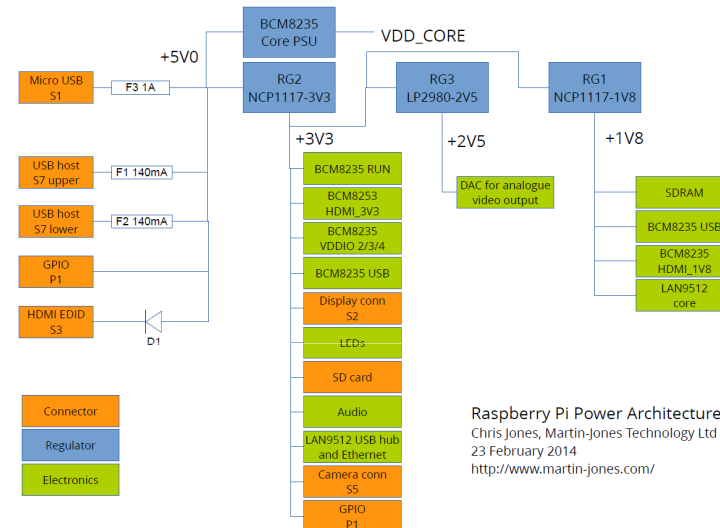
40 ans de microcontrôleurs

Apple II (1977) ou Smaky



Proc: 1Mhz, 8bits, 4kRAM, 8kROM, 24x40char, Cassette, HP , 8 Ports, Composite vidéo

Raspberry II (2015)



Raspberry Pi Power Architecture
Chris Jones, Martin-Jones Technology Ltd
23 February 2014
<http://www.martin-jones.com/>



Proc: 900MHz, 4Core, 32bits, 1GB RAM, HDMI 40 GPIO, 4USB, ETH, SD, AUDIO/VIDEO

Comparaison de microcontrôleurs

Année/Produits	Transistors	Clock	Mem(max)
74 Intel 8080	4'500	2-4MHz	64kB adressable
75 MOS Tech. 6502	3'510	1MHz	Très populaire, KIM1, PET, Smaky
76 Zilog Z80	8'500	2,5-8	Sinclair, TRS-80, Amstrad, Smaky Pro
77 Motorola 68000	68'000	~10MHz	8/16MB 1MIPS
79 Intel 8051	50'000	12MHz	4k+128 on chip
85 Acorn ARM1	25'000	8MHz?	2MB Ram on chip
93 PIC 16C84	?50'000?	2.5MIPS	1kB ROM + 68bytes Ram *1 ^{er} flash
96 AVR Tiny/Mega		20MIPS	0.5-256kB
00 Pentium 4	42'000'000	>1.3GHz	4GB
01 PIC 24F		16/40MIPS	4k-256k (512k)
07 PIC32MX		105DMIPS	512kB+128kRam
08 AVR 32		91DMIPS	16-512kB
13 SAMD (Atmel) Cortex M0+		45DMIPS	16-256kB
15 Cortex M4 (ST)	26'000'000	180DMIPS	<2MBFlash on chip
11 Quad-Core i7	995'000'000	>2.8GHz	16GB
15 SPARC M7	10'000'000'000	3.6GHz	4-16cores, 64MB cache

Source: https://en.wikipedia.org/wiki/Transistor_count

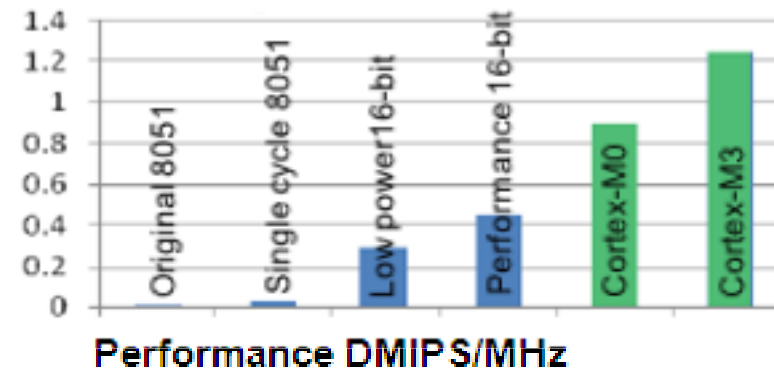
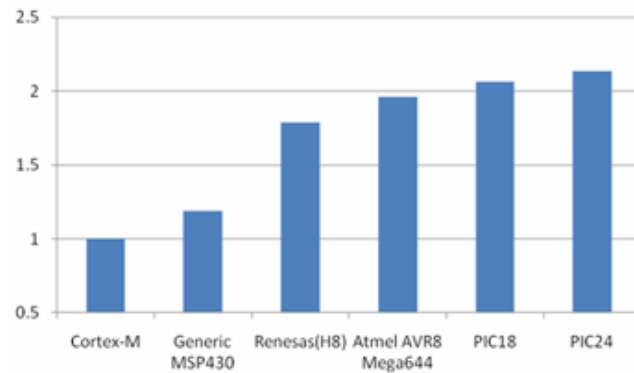
Comparaison

8051, PIC, AVR, MSP430, ARM

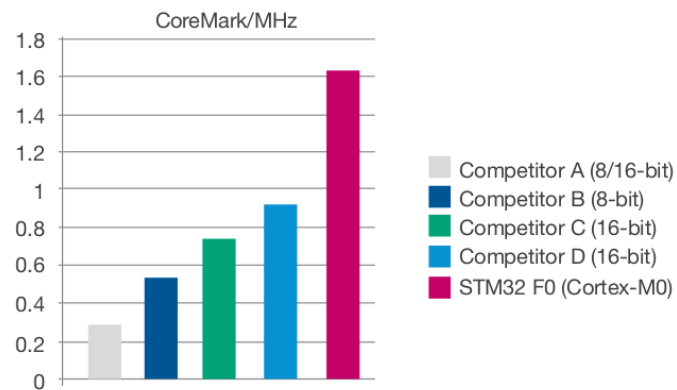
- Différentes architectures = vitesse
- Architecture 8bit pour la base, 16/32bits pour ARM
- 8051, AVR, ARM peuvent adresser toute la RAM, PIC que des blocs de 256 bytes.
- 8051 et PIC, MSP utilisent plusieurs clocks /instr., AVR et ARM 1 cycle/instr.
- AVR peuvent remplacer les 8051, ils sont similaires
- AVR+ARM ont des compilateurs efficaces + GCC
- MSP a une consommation ultra-basse => application ou l'on veut changer de batterie 1x par ans
- ARM = Puissance, recommandé pour du multi-tâche moyen et haut niveau.

Comparaison ARM, autres uC

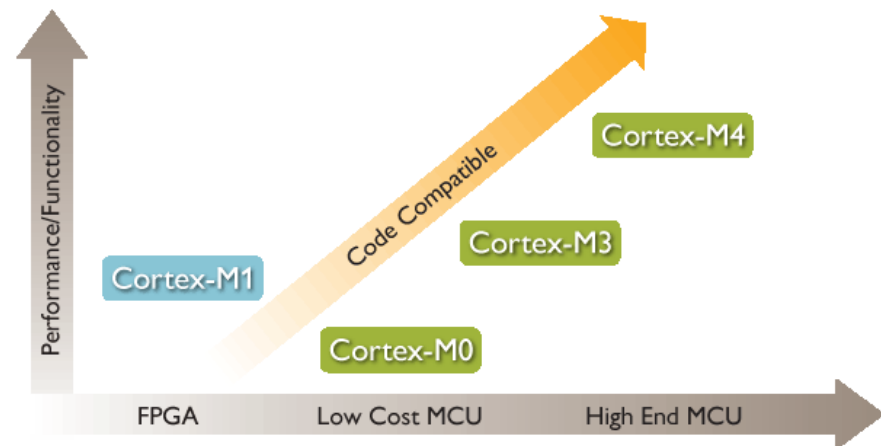
Average code size



STM32 F0 Benchmark Positioning



The STM32 F0 provides an optimal balance of cost, performance, and peripherals for embedded applications.



Efficacité ARM comparé au 8 bits

Comparing
16-bit
multiply
operations
across
processor
architectures

8-bit example	16-bit example	ARM Cortex-M
MOV A, XL ; 2 bytes MOV B, YL ; 3 bytes MUL AB; 1 byte MOV R0, A; 1 byte MOV R1, B; 3 bytes MOV A, XL ; 2 bytes MOV B, YH ; 3 bytes MUL AB; 1 byte ADD A, R1; 1 byte MOV R1, A; 1 byte MOV A, B ; 2 bytes ADDC A, #0 ; 2 bytes MOV R2, A; 1 byte MOV A, XH ; 2 bytes MOV B, YL ; 3 bytes	MUL AB; 1 byte ADD A, R1; 1 byte MOV R1, A; 1 byte MOV A, B ; 2 bytes ADDC A, R2 ; 1 bytes MOV R2, A; 1 byte MOV A, XH ; 2 bytes MOV B, YH ; 3 bytes MUL AB; 1 byte ADD A, R2; 1 byte MOV R2, A; 1 byte MOV A, B ; 2 bytes ADDC A, #0 ; 2 bytes MOV R3, A; 1 byte	MOV R4,&0130h MOV R5,&0138h MOV SumLo,R6 MOV SumHi,R7 (Operands are moved to and from a memory mapped hardware multiply unit)
		MULS r0,r1,r0

N.B. The Cortex-M multiply in fact performs a 32-bit multiply, here we assume r0 and r1 contain 16-bit data.

	ARM	8051	AVR	PIC	MSP430
Bus Width	32-bit mostly also available in 64-bit	8-bit for standard core	8/32-bit	8/16/32-bit	16-bit
Communication Protocols	UART, USART, LIN, I2C, SPI, CAN, USB, Ethernet, I2S, DSP, SAI (serial audio interface), IrDA	UART, USART, SPI, I2C	UART, USART, SPI, I2C, (special purpose AVR support CAN, USB, Ethernet)	PCI, UART, USART, LIN, CAN, Ethernet, SPI, I2S	UART, USART, LIN, I2C, SPI, I2S, IrDA
Speed	1 clock / instruction cycle	12 clock / instruction cycle	1 clock / instruction cycle	4 clock / instruction cycle	6 clock / instruction cycle
Memory	Flash, SDRAM, EEPROM	ROM, SRAM, FLASH	Flash, SRAM, EEPROM	SRAM, FLASH	SRAM, FLASH
ISA	RISC	CISC	RISC	Some feature of RISC	Some feature of RISC
Memory Architecture	Modified Harvard architecture (M3-M4-M7)	Von Neumann architecture	Modified Harvard	Harvard architecture	Von Neumann architecture
Power Consumption	Low	Average	Low	Low	Ultra Low
Families	ARMv4,5,6,7 and Cortex series	8051 variants	Tiny, Atmega, Xmega, special purpose AVR,	PIC16, PIC17, PIC18, PIC24, PIC32	MSP430X, MSP430FR57xx, MSP430x1xx to x6xx series
Community	Vast	Vast	Very Good	Very Good	Average
Manufacturer	STM32F030 1000=0.96	NXP, Atmel, Silicon Labs, Dallas, Cypress, infineon, etc	ATMEGA328 1000=1.68	PIC18f4550 1200=4.39	TI
Cost (as compared to feature provided)	Low	Very Low	Average	Average	Average
Other Feature	32bit 48MHz 64k/8k	known for its Standard	8bit 20MHz 32k/2k	8bit 12MHz 32k/2k	known for Ultra low power operation
Popular Microcontrollers	LP ARM Cortex-M7, etc	AT89C51, P89v51, etc	Community	PIC32MXXX	MSP430G2553, MSP430 launchpad.

ARM Origine

- ARM= **A**dvanced **R**isc **M**achine
- Joint venture entre Acorn, Apple, VLSI en 1990
- Leader dans l'architecture de processeurs 16/32bits.
- **ARM ne fabrique pas de chip**
- La société licencie ces processeurs haute performance, bas prix, efficace en consommation aux compagnies leaders de l'électronique tels que ST, ATMEL, NXP, Cirrus Logic etc.
- Utilisation principale, appareils portables (tablettes, smartphones), robotique, automation, grand public

Appareils portables



Apple iPhone (3s)
ARM11



Motorola Z8 Smart
phone
ARM11

Appareils divers



Blackberry
ARM11



Nokia E90 Communicator
ARM11

Marché des processeurs ARM

A: Haute performance: Automobile, Tablettes, Multimédia, Téléphonie

R: Temps réel: Automobile, Disques dur, Téléphonie, Routeurs

M: Basse consommation: Appareils portables, bas prix.

La gamme ARM-M

M0: Alternative pour les processeurs 8 bits, processeur 32 bits très basse consommation.

M1: Implémentés dans des processeurs FPGA

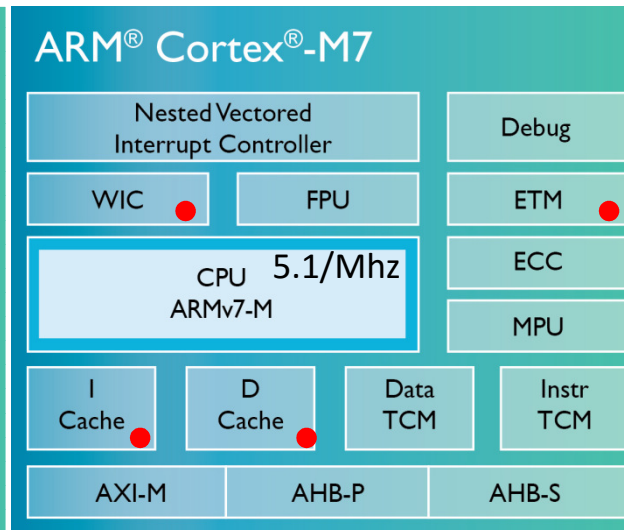
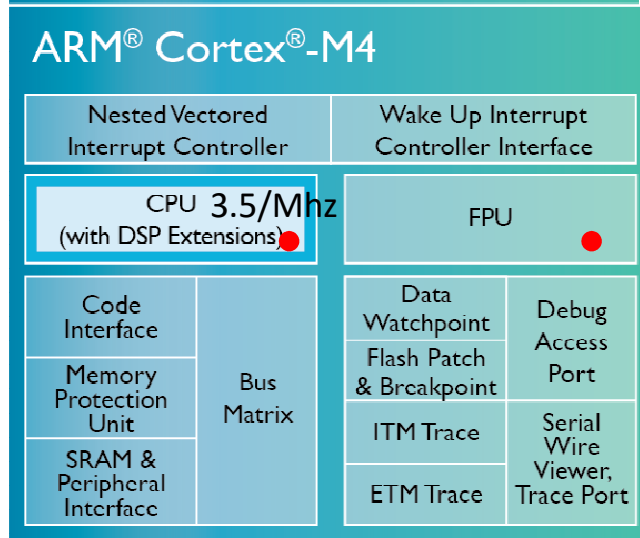
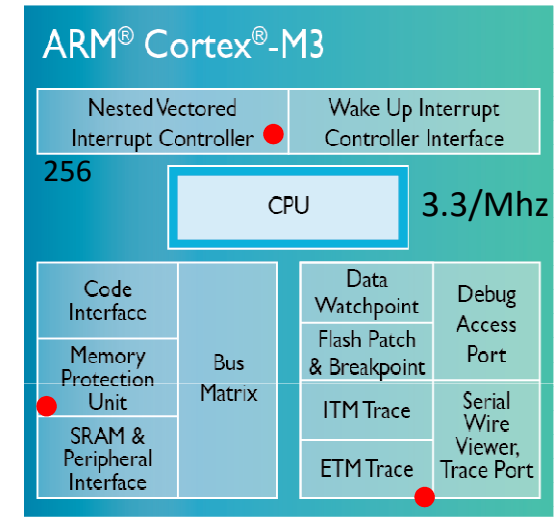
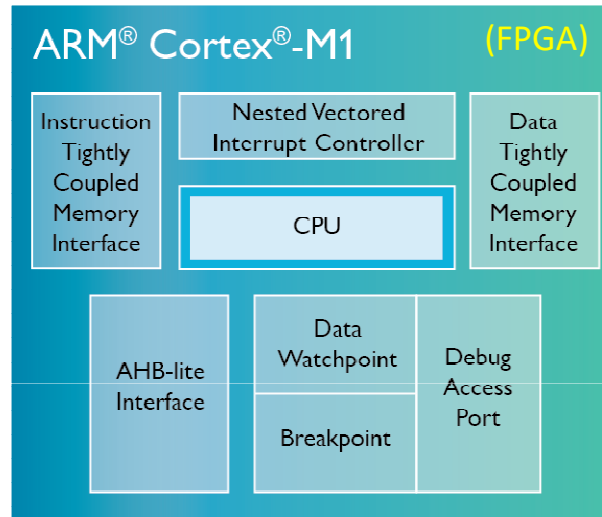
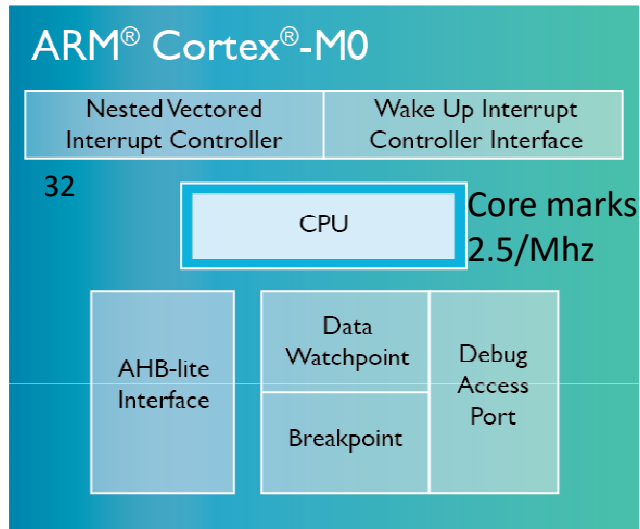
M3: Haute performance à basse consommation

M4: Processeur avec fonction DSP, optimisation mathématiques et virgule flottante simple précision.

M7: Processeur très haute performance

- Les processeurs de la famille 7,9,11 sont encore intégrés dans beaucoup d'appareils, mais migrent vers les processeurs M0 et M3

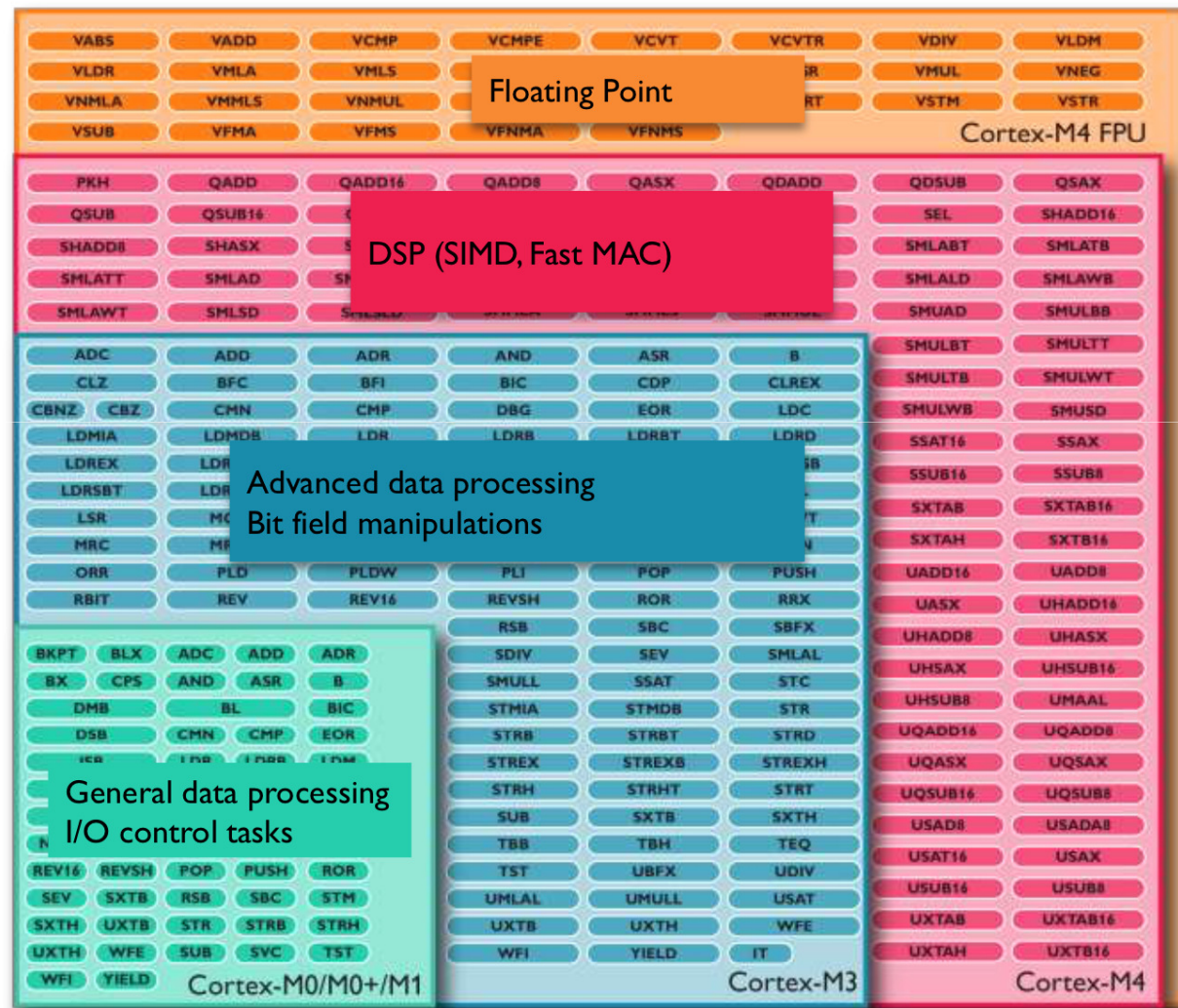
ARM Les processeurs gamme M



DSP= Digital Signal Processing
 FPU= Floating Point Unit
 AHB= Advanced High Performance Bus
 WIC=Weak-up interrupt controller
 ETM=Embedded Trace Macrocell™
 ECC=Error correction coding
 MPU=Memory Protection Unit
 TMC=Tightly-Coupled Memory
 AXI=Advanced eXtensible Interface

ARM Cortex-M Instruction Set Architecture

Cortex-M0/M0+



ARM Résumé 1

- ARM RISC (Reduced instruction set) avec quelques amendements pour satisfaire les systèmes embarqués.
- Large Registres (R0 à R16) contraire à RISC
- Architecture LOAD et STORE (LDR/STR R0->adresse)
- Instructions uniformes à longueur fixe
- Processeur 32 bits
- Bon rapport performance/consommation
- Haute densité de code
- Principalement des instructions à 1 cycle
- Vitesse de 1MHz à 1.25 GHz

ARM Résumé 2

- Principalement utilisé dans des systèmes embarqués.
- ARM a un bon support pour RTOS, Linux, QNX, VxWorks, FreeRTOS, ...
- ARM est connu pour son excellente performance à basse consommation
- ARM7TDMI a le plus de succès avec 1Millions de produits vendu par trimestre.

Outils de développement

IDE, compiler, linker, debugger, flashing (in alphabetical order):

- **Atmel Studio**^[note 1] by Atmel^[1] (based on Visual Studio^[2] and GNU GCC Toolchain^[3])
- **Atollic TrueSTUDIO** by Atollic AB^[4]
- Code Composer Studio^[note 2] by Texas Instruments^[5]
- **CoIDE** by CoCoX^[6]
- CrossWorks for ARM by Rowley^[7]
- Dave by Infineon. For XMC processors only. Includes project wizard, detailed register decoding and a code library still under development.^[8]
- DRT by SOMNIUM Technologies^[9]
- Eclipse as IDE, with GNU Tools as compiler/linker, e.g. aided with GNU ARM Eclipse Plug-ins^[10]
- **Em::Blocks - free (non-eclipse) IDE for STlink** (live data updates), openOCD, including GNU Tools for ARM and project wizards for ST, Atmel, EnergyMicro etc.^[11]
- emIDE - free Visual Studio Style IDE including GNU Tools for ARM^[12]
- GNU Tools (aka GCC) for ARM Embedded Processors by ARM Ltd - free GCC for bare meta^{[13][14]}
- IAR Embedded Workbench for ARM by IAR^[15]
- ICC by ImageCraft^[16]
- **Keil MDK-ARM** by Keil^[17]
- LPCpresso^[note 3] by NXP^[18] (formerly Red Suite by Code Red Technologies^[19])
- MikroC - [mikroC](#)
- MicroEJ - Platforming for both Java and C ^[3]
- Ride and RKit for ARM by Raisonance^[20]
- Sourcery CodeBench by Mentor Graphics^[21]
- TASKING VX-Toolset by Altium^[22]
- Visual Studio by Microsoft as IDE, with GNU Tools as compiler/linker - e.g. supported by VisualGDB^[24]
- winIDEA/winIDEAOpen by iSYSTEM^[25]
- YAGARTO - free GCC (no longer supported)^[26]

IDE for Arduino ARM boards

- Arduino^[note 4] - IDE for Atmel SAM3X ([Arduino Due](#))
- Energia - Arduino IDE for [Texas Instruments Tiva](#) and CC3200

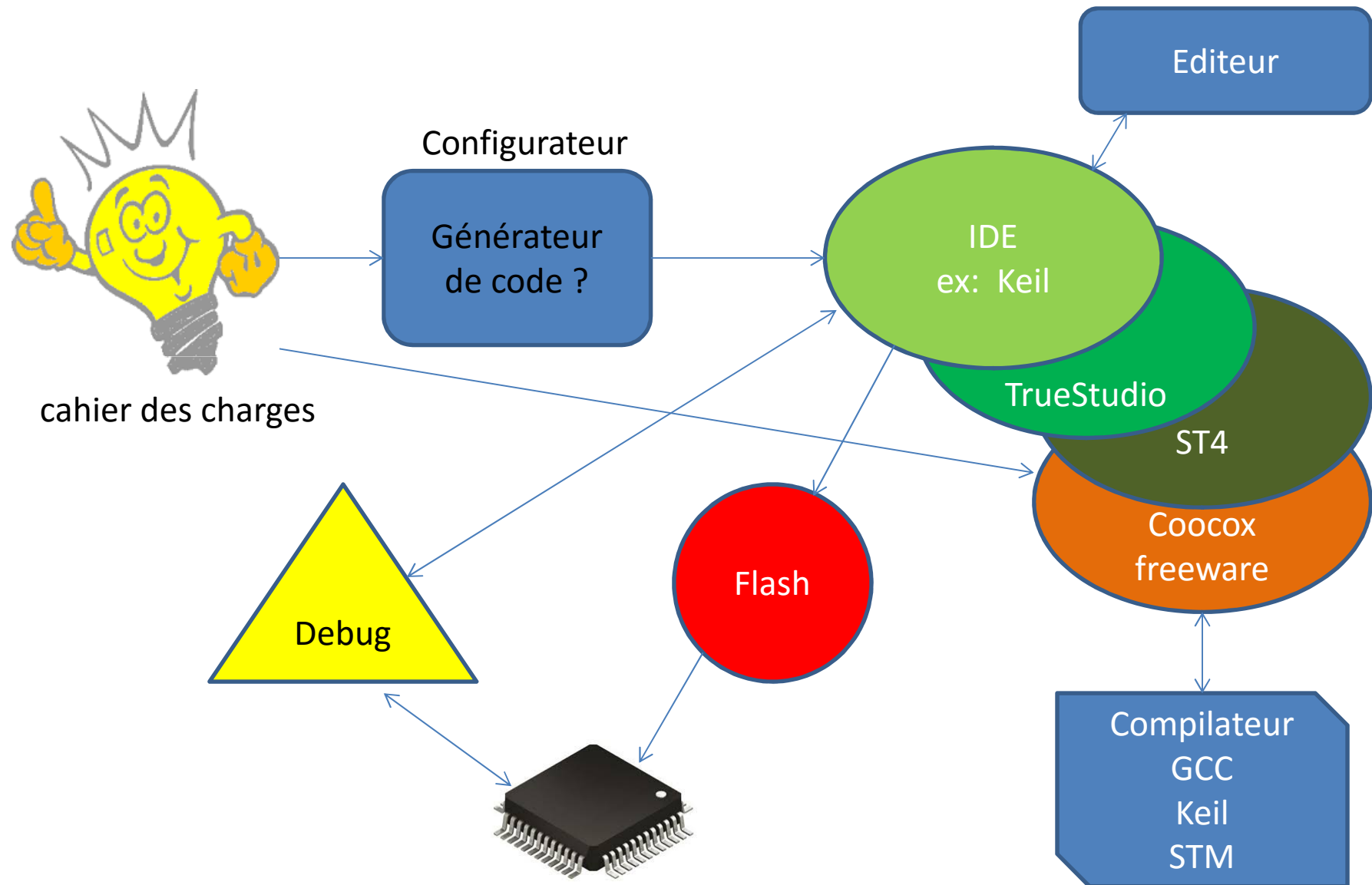


ARM Outils de développement

- Propriétaires
 - KEIL, IAR Workbench
- Open-Source
 - GNUARM / GCC
 - Yagarto
- IDE (env. de développement intégré) free
 - Eclipse
 - Coocox
 - EM:Blocs



Nouveau projet avec Cube(STM)



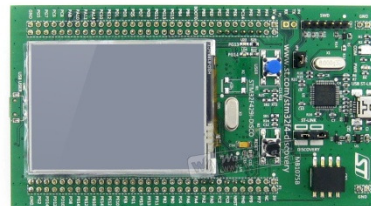
Carte, processeur Un choix énorme



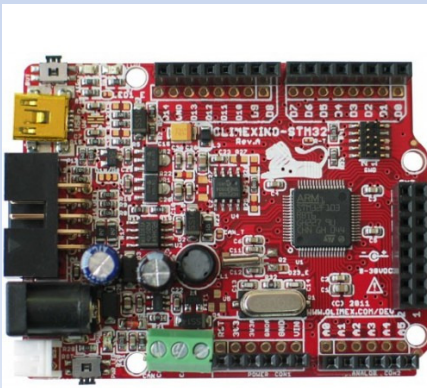
STM32F407 disco 15chf



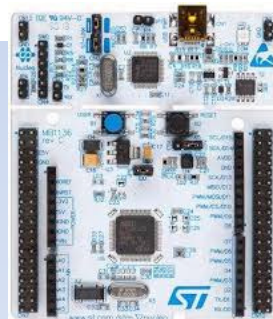
STM32 Low-Power 20€



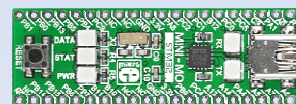
STM32F429 disco1 31chf



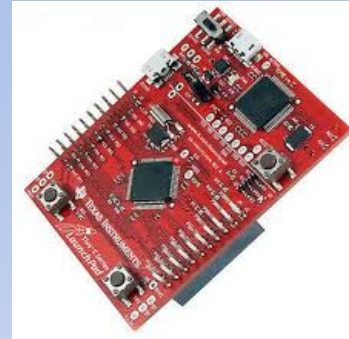
Olimexino STM32 30chf
With bootloader!!!



Nucleo-F4 11chf



MicroE STM32 30chf

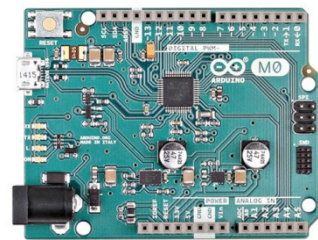


Low-Power Stellaris TI 11chf



Nucleo-32 F0 10chf
On board ST-Link

Arduino M0 35chf



Programmer
Debugger



ST-Link

Logiciel

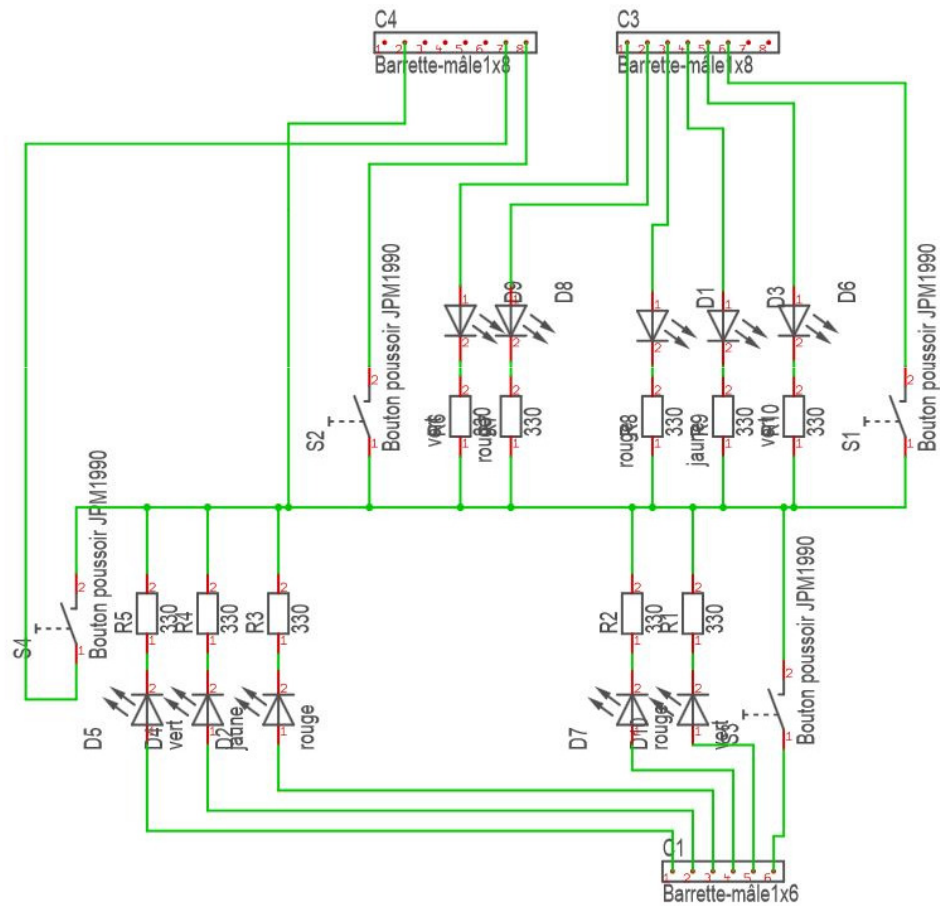
- IDE Coocox (libre) avec compilateur GCC
- IDE KEIL (payant >32k)
 - compilateur GCC et ou Keil (intégrés)
- Générateur de code CUBE (STM) libre
- Pour des cartes ARM Atmel, l'IDE Atmel Studio intègre également un compilateur libre.



Premier pas

- Analyser le schéma, définir les pins entrée/sorties
- Comprendre comment définir les IO sur ARM
- Comprendre comment générer un timer pour les délais
- Essai avec Keil + Cube
- Essai avec Coocox

Franic Traffic

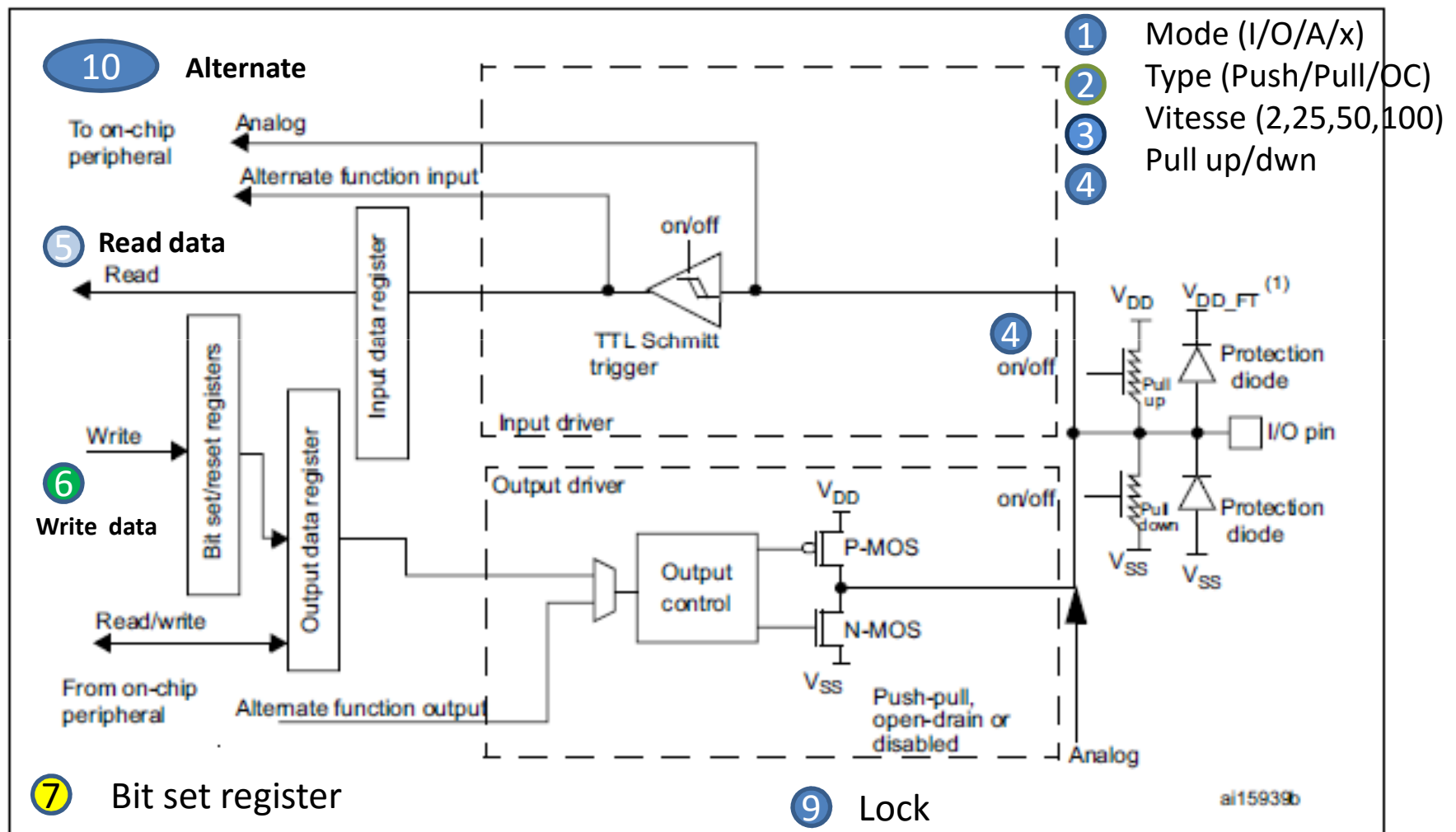


Connections

DET1	D10	I
F1R	B12	O
F1J	B14	O
F1V	D8	O
P1R	B10	O
P1V	E14	O
BP1	E12	I
DET2	E10	I
F2R	A15	O
F2J	C11	O
F2V	D0	O
P2R	A8	O
P2V	C8	O
BP2	C6	I

Fonction timer
Pour les délais

Structure GPIO



Registres définissant les entrées/sorties

```
typedef struct {  
    __IO uint32_t MODER; /*!< GPIO port mode register, Address offset: 0x00 */  
    __IO uint32_t OTYPER; /*!< GPIO port output type register, Address offset: 0x04 */  
    __IO uint32_t OSPEEDR; /*!< GPIO port output speed register, Address offset: 0x08 */  
    __IO uint32_t PUPDR; /*!< GPIO port pull-up/pull-down register, Address offset: 0x0C */  
    __IO uint32_t IDR; /*!< GPIO port input data register, Address offset: 0x10 */  
    __IO uint32_t ODR; /*!< GPIO port output data register, Address offset: 0x14 */  
    __IO uint16_t BSRRL; /*!< GPIO port bit set/reset low register, Address offset: 0x18 */  
    __IO uint16_t BSRRH; /*!< GPIO port bit set/reset high register, Address offset: 0x1A */  
    __IO uint32_t LCKR; /*!< GPIO port configuration lock register, Address offset: 0x1C */  
    __IO uint32_t AFR[2]; /*!< GPIO alternate function registers, Address offset: 0x24-0x28 */ }  
    GPIO_TypeDef;
```

Utilisation de structures

exemple pour définir les GPIO

```
01.#include "stm32f4xx_rcc.h"
02.#include "stm32f4xx_gpio.h"
03.
04.int main()
05. {
06.// Initialise the peripheral clock.
07.    RCC->AHB1ENR |= RCC_AHB1Periph_GPIOD; // crée une structure IO
08.// Initilaise the GPIO port.
09.    GPIOD->MODER |= GPIO_Mode_OUT;
10.    GPIOD->OSPEEDR |= GPIO_Speed_25MHz;
11.    GPIOD->OTYPER |= GPIO_OType_PP;
12.    GPIOD->PUPDR |= GPIO_PuPd_NOPULL;
13.// Toggle Port D, pin 0 indefinitely.
14. while (1)
15.     {
16.        GPIOD->BSRRL = GPIO_Pin_0; // exemple pin 0 est mis à 1 (clear)
17.        GPIOD->BSRRH = GPIO_Pin_0; // exemple pin 0 est mis à 0 (set)
18.    }
19. }
```

Fonctions de la librairie stm32f0xx_gpio.h

/* GPIO Read and Write functions ***/**

uint8_t GPIO_ReadInputDataBit(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin);

uint16_t GPIO_ReadInputData(GPIO_TypeDef* GPIOx);

uint8_t GPIO_ReadOutputDataBit(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin);

uint16_t GPIO_ReadOutputData(GPIO_TypeDef* GPIOx);

void GPIO_SetBits(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin);

void GPIO_ResetBits(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin);

void GPIO_WriteBit(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin, BitAction BitVal);

void GPIO_Write(GPIO_TypeDef* GPIOx, uint16_t PortVal);

Utilisation de la fonction systick

Exemple pour définir un délai (ms)

```
// dans main
if (SysTick_Config(SystemCoreClock / 1000)) // définit systick = 1ms
{
    /* Capture error */
    while (1);
}
```

```
void Delay(__IO uint32_t nTime)
{
    TimingDelay = nTime;
    while(TimingDelay != 0);
}
```

```
void TimingDelay_Decrement(void) // routine appelée par une routine système
{
    if (TimingDelay != 0x00){
        TimingDelay--; // decrement every 1ms
    }
}
```

IDE Coocox

Configurateur + Compilateur

Demo

Configurateur,

Utilitaire aidant à préparer le code de base pour un nouveau projet.

CooCox CoIDE --- D:/CooCox/CoIDE/workspace/libtest/libtest.coproj

File Edit View Project Flash Debug Search Help

Components [STM32F407VG 1]

- Device [STM32F407VG 1]
- Common
 - M4 CMSIS Core (with 9 examples)
- Boot
 - CMSIS BOOT (with 8 examples)
- Peripheral.ST
 - RCC (with 13 examples)
 - GPIO (with 17 examples)

Project [libtest]

Target [libtest]

- libtest
 - cmsis
 - core_cm4_simd.h
 - core_cm4.h
 - core_cmFunc.h
 - core_cmInstr.h
 - cmsis_boot
 - startup
 - startup_stm32f4xx.c
 - stm32f4xx_conf.h
 - stm32f4xx.h
 - system_stm32f4xx.c
 - system_stm32f4xx.h
 - cmsis_lib
 - include
 - stm32f4xx_gpio.h
 - stm32f4xx_rcc.h
 - source

Step 3 Select Basic Components [ST / STM32F407VG]

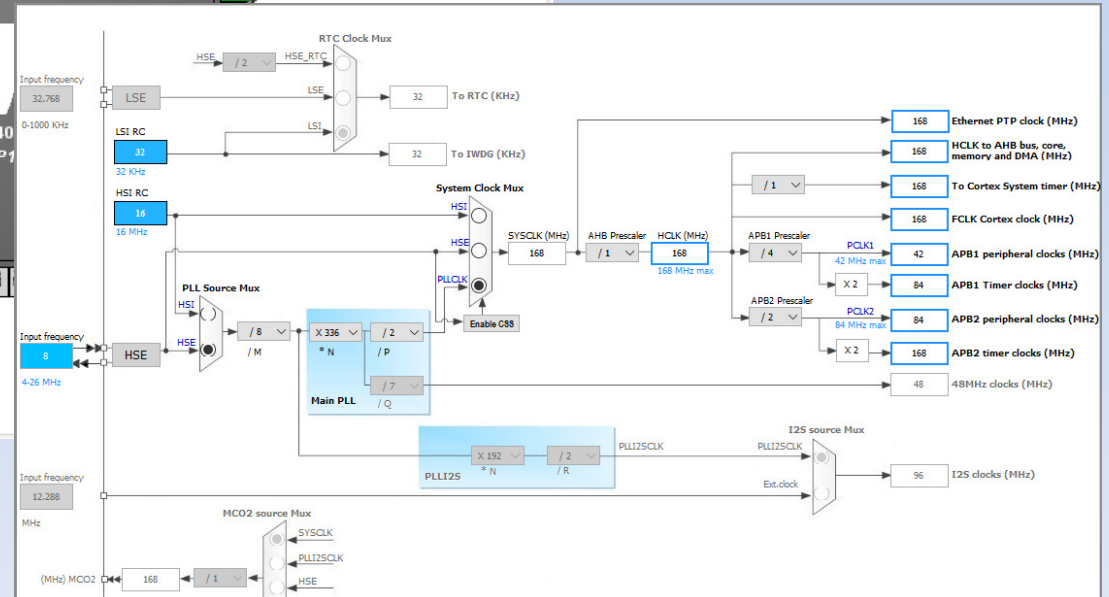
Category	Component	Description	Status	Source
COMMON	☐ C Library	Implement the minimal functionality required to allow newlib to l...	Available	CooCox
	☐ Retarget printf	Implementation of printf(), sprintf() to reduce memory footprint	Available	CooCox
	☐ Semihosting	Implementation of Semihosting GetChar/SendChar	Available	CooCox
	☒ M4 CMSIS Core	CMSIS core for Cortex M4 V 3.01	Available	CooCox
	☐ Retarget printf, syscalls	SYSCALLS, retarget printf	Download	thaon (Au
	☐ METABuffer	Applies a FIFO or LIFO behavior to any buffer of any kind of elem...	Available	Kairos (Au
	☐ TimeOut	Object that implement a timeout mechanism based on signal/ca...	Available	Kairos (Au
BOOT	☒ CMSIS BOOT	CMSIS BOOT for STM32F4x series	Available	CooCox
PERIPHERAL.ST	☒ RCC	Reset and clock control for STM32F4xx	Available	CooCox
	☐ PWR	System configuration controller for STM32F4xx	Available	CooCox
	☐ CRC	Cyclic Redundancy Check for STM32F4xx	Available	CooCox
	☒ GPIO	General Purpose Input/Output for STM32F4xx	Available	CooCox
	☐ EXTI	External Interrupt/Line Controller for STM32F4xx	Available	CooCox
	☐ RTC	Real Time Clock for STM32F4xx	Available	CooCox
	☐ IWDG	Independent watchdog for STM32F4xx	Available	CooCox
	☐ WWDG	Window watchdog for STM32F4xx	Available	CooCox
	☐ SPI	Serial peripheral interface for STM32F4xx	Available	CooCox
	☐ I2C	Inter-integrated circuit interface for STM32F4xx	Available	CooCox
	☐ DMA	Direct Memory Access for STM32F4xx	Available	CooCox
	☐ FLASH	Flash Memory Controller for STM32F4xx	Available	CooCox
	☐ TIM	Advanced-control timers for STM32F4xx	Available	CooCox
	☐ ADC	Analog/Digital Convert for STM32F4xx	Available	CooCox
	☐ [Blue Arrow]	General Purpose Input/Output for STM32F4xx	Available	CooCox
	☐ [Green Arrow]	Static memory controller for STM32F4xx	Available	CooCox
	☐ [Orange Arrow]	Universal asynchronous receiver transmitter for STM32...	Available	CooCox
	☐ RNG	Random number generator for STM32F4xx	Available	CooCox

Manufacturers Chips Peripherals Drivers Others

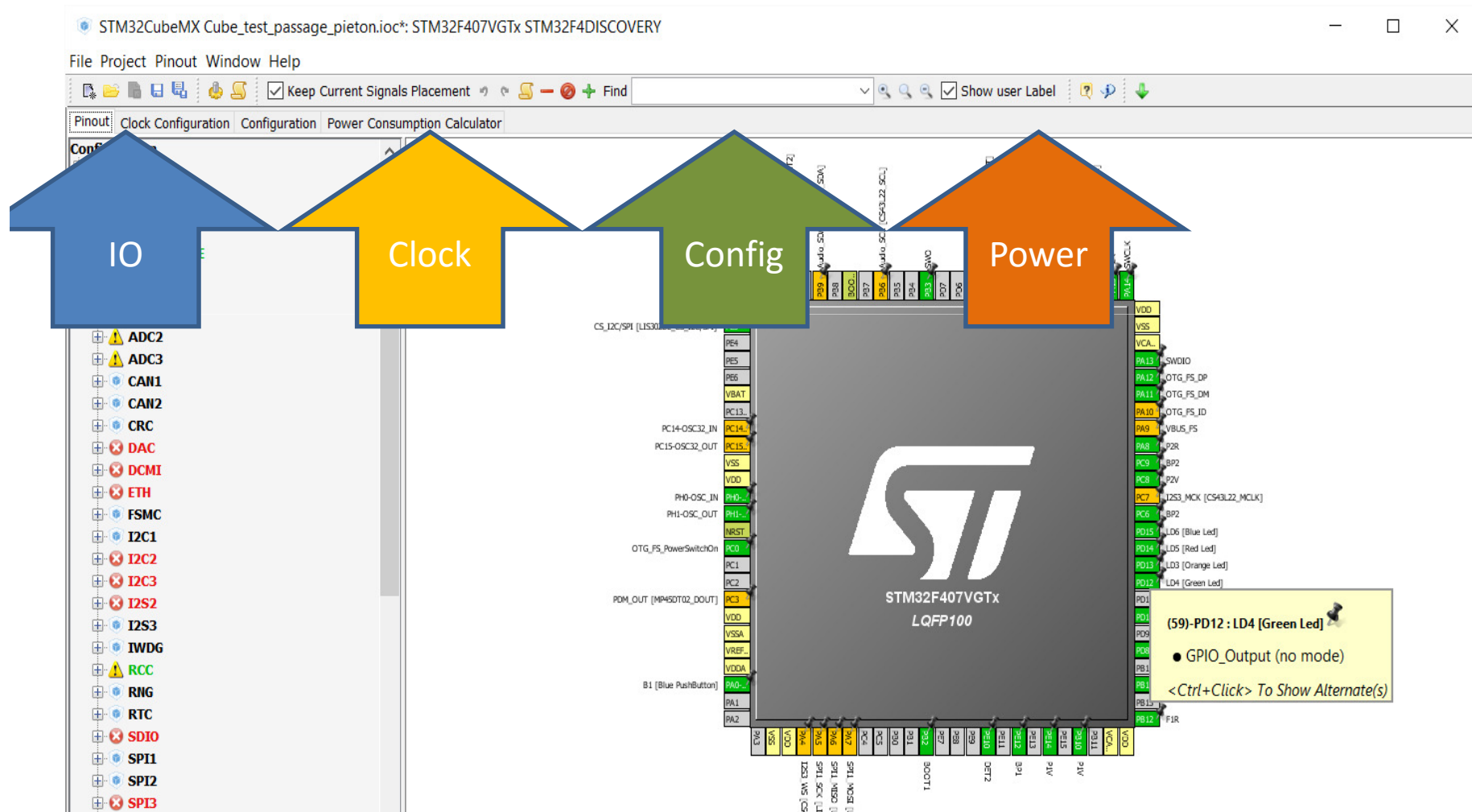
Environnement ST->KEIL

Demo

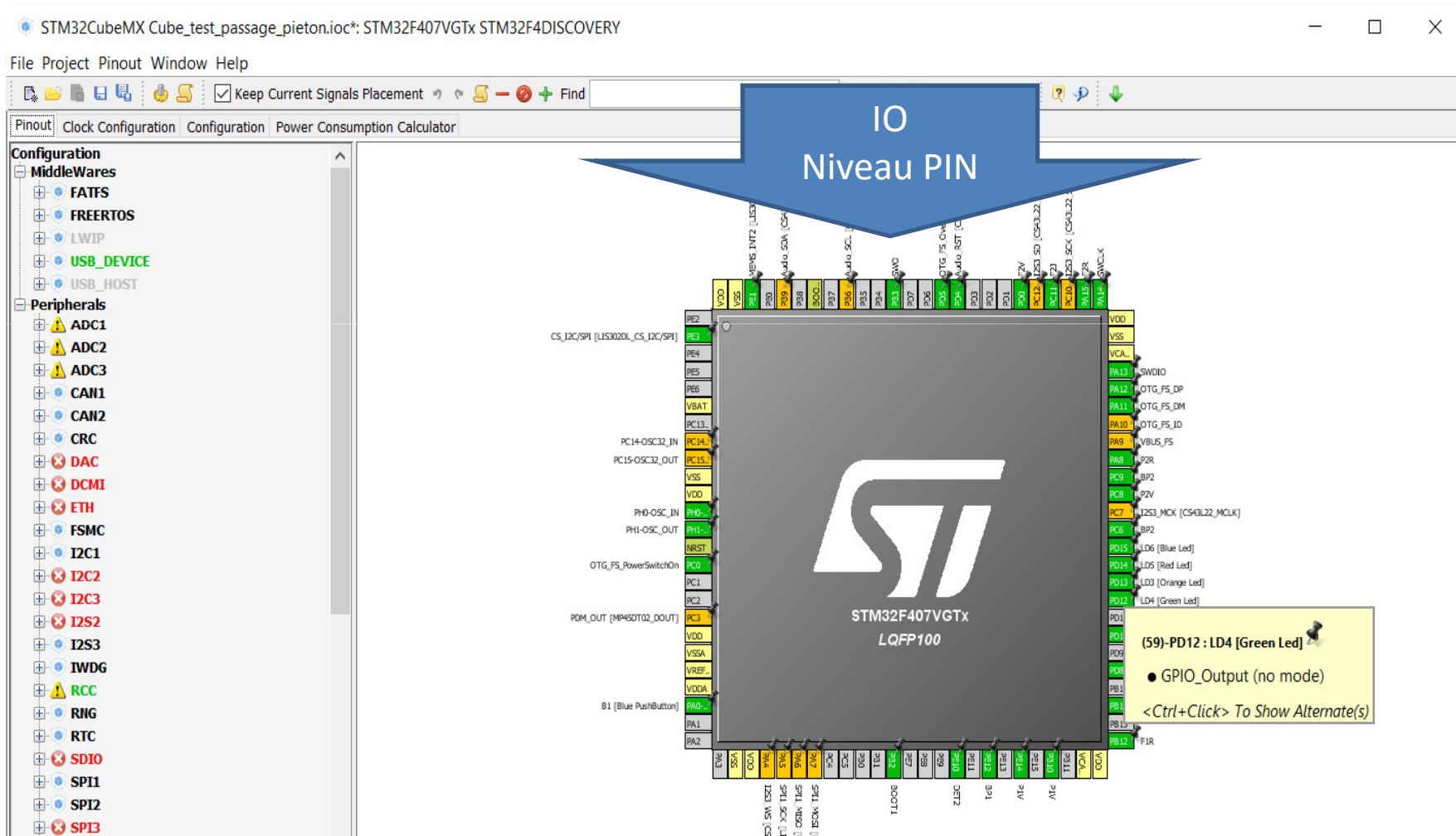
Configurateur STM



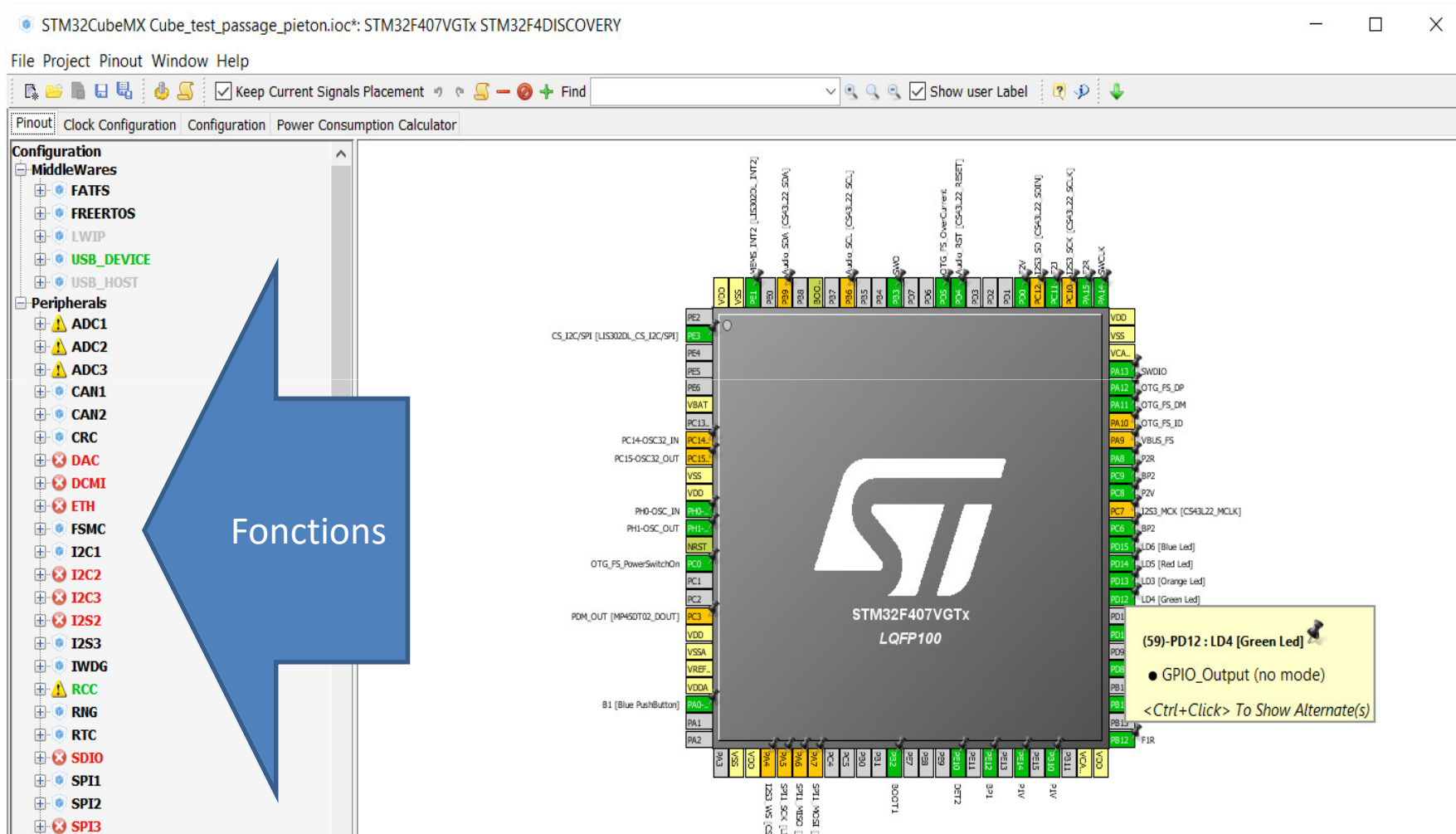
Cube, Configurateur/Générateur



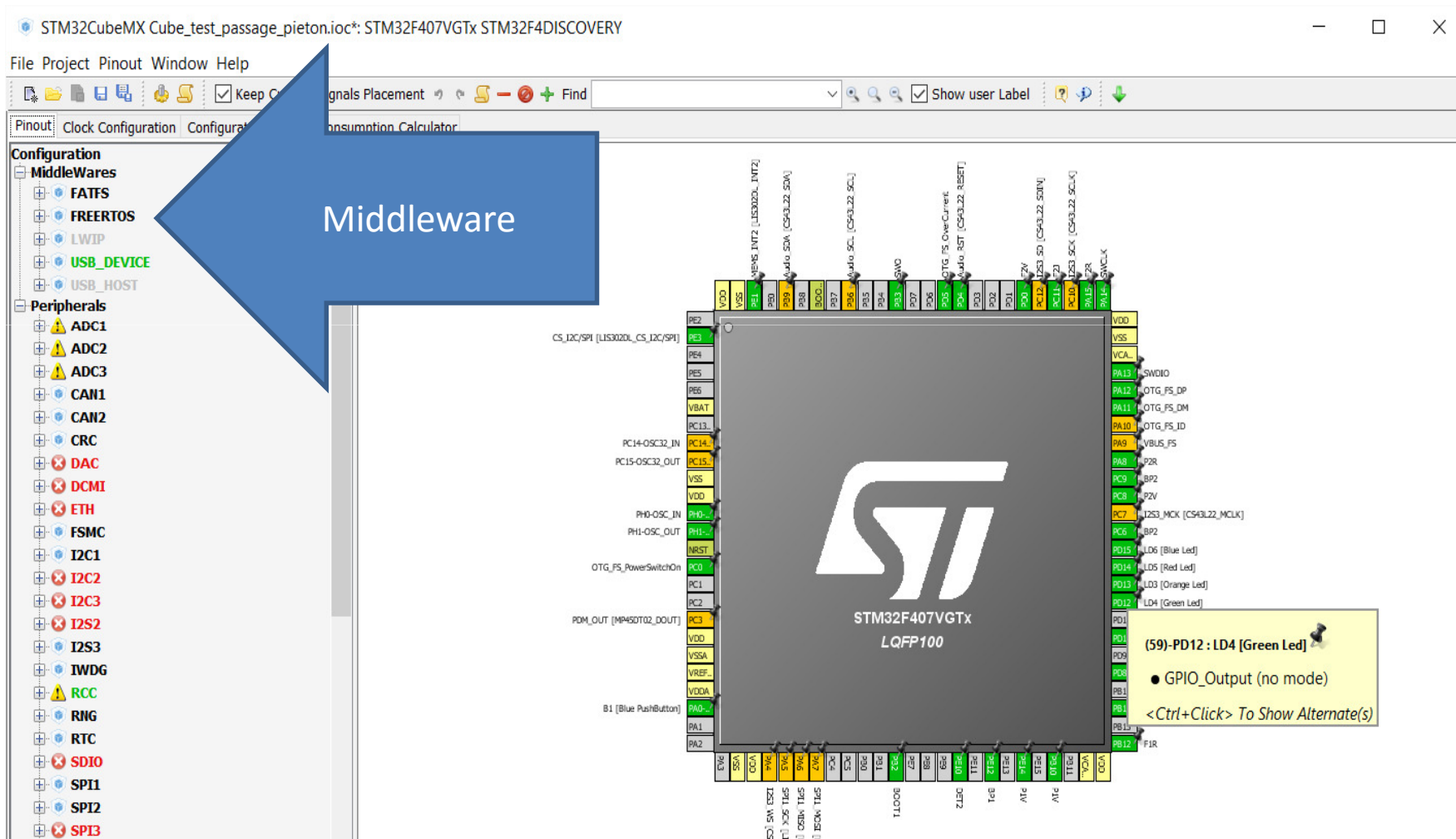
Cube, Configurateur/Générateur



Cube, Configurateur/Générateur

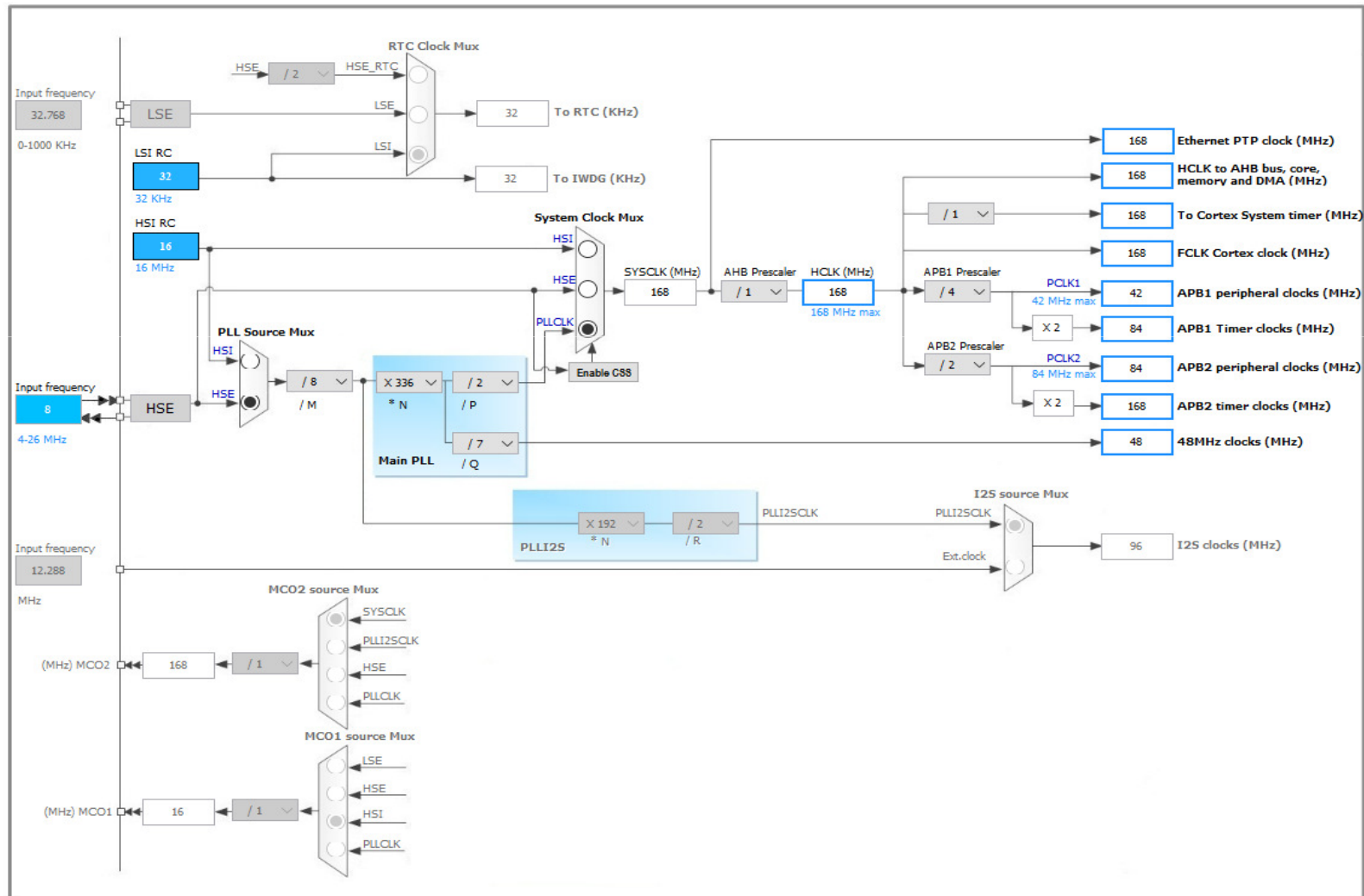


Cube, Configurateur/Générateur

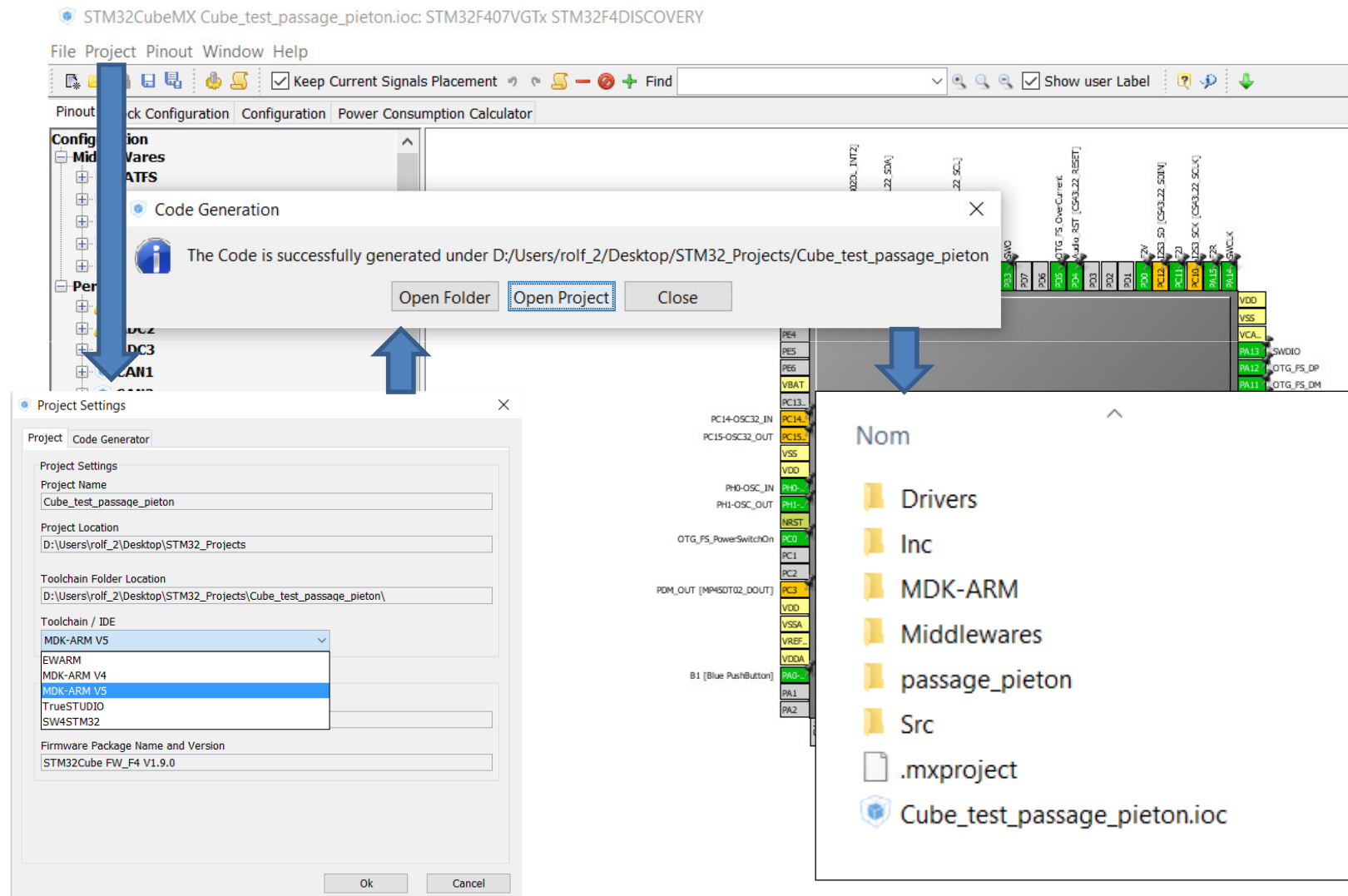


Cube Clock Configurator

génère code initialisation

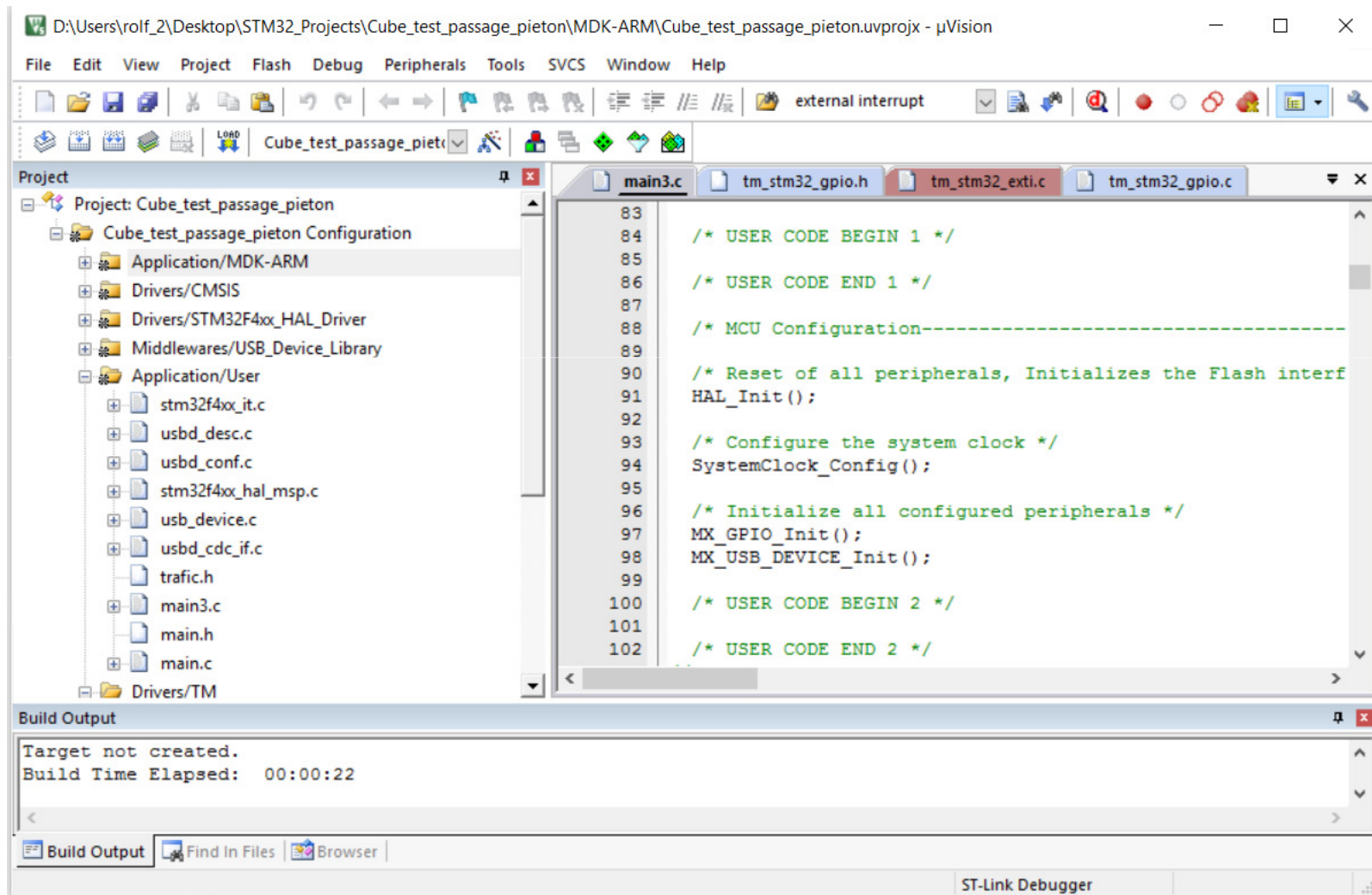


Cube générer code



Compilateur KEIL

Standard des pros....\$\$\$\$



Gratuit pour Processeur F0 ou code < 32kB

Librairies (ST)

- La complexité des processeurs ARM, multiplication des registres et flags a poussé à l'abstraction des fonctions et registres. Nous trouverons donc des fichiers de définitions et de routines permettant de configurer et programmer le niveau matériel avec plus de compréhension et de confort.
- **CMSIS (Cortex Microcontrôleur Software Interface Standard)** niveau ASM, contient les définitions pour les Perif. Func. Instr.
- **CMSIS Boot (Cortex Processor Start code)** Init Clock, IO, Stack,...
- **Standard Peripheral Library** (niveau PORT, PIN, GPIO Init, Set, Clear,...) un fichier par fonction (UART, GPIO, I2C,...).

Extensions

- **HAL (Hardware Abstraction Layer)** Librairie focus plus sur les fonctions de périphériques que sur le HW.
- **TM (Tilen Majerle HAL library)** Extension de la librairie HAL avec de nombreux pilotes matériel.
-

Différences des librairies

Librairie STM (CMSIS) standard

```
RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOG, ENABLE);
GPIO_InitTypeDef GPIO_InitStructure;
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_12 | GPIO_Pin_13 | GPIO_Pin_14 | GPIO_Pin_15;
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF;
GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL;
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_100MHz;
GPIO_Init(GPIOD, &GPIO_InitStructure);
```

```
GPIO_ToggleBits (GPIOD, GPIO_Pin_5);
```

Librairie HAL (dependant du constructeur)

```
GPIO_InitTypeDef GPIO_InitStructure;
__GPIOD_CLK_ENABLE();
```

```
GPIO_InitStructure.Pin = GPIO_PIN_12;
```

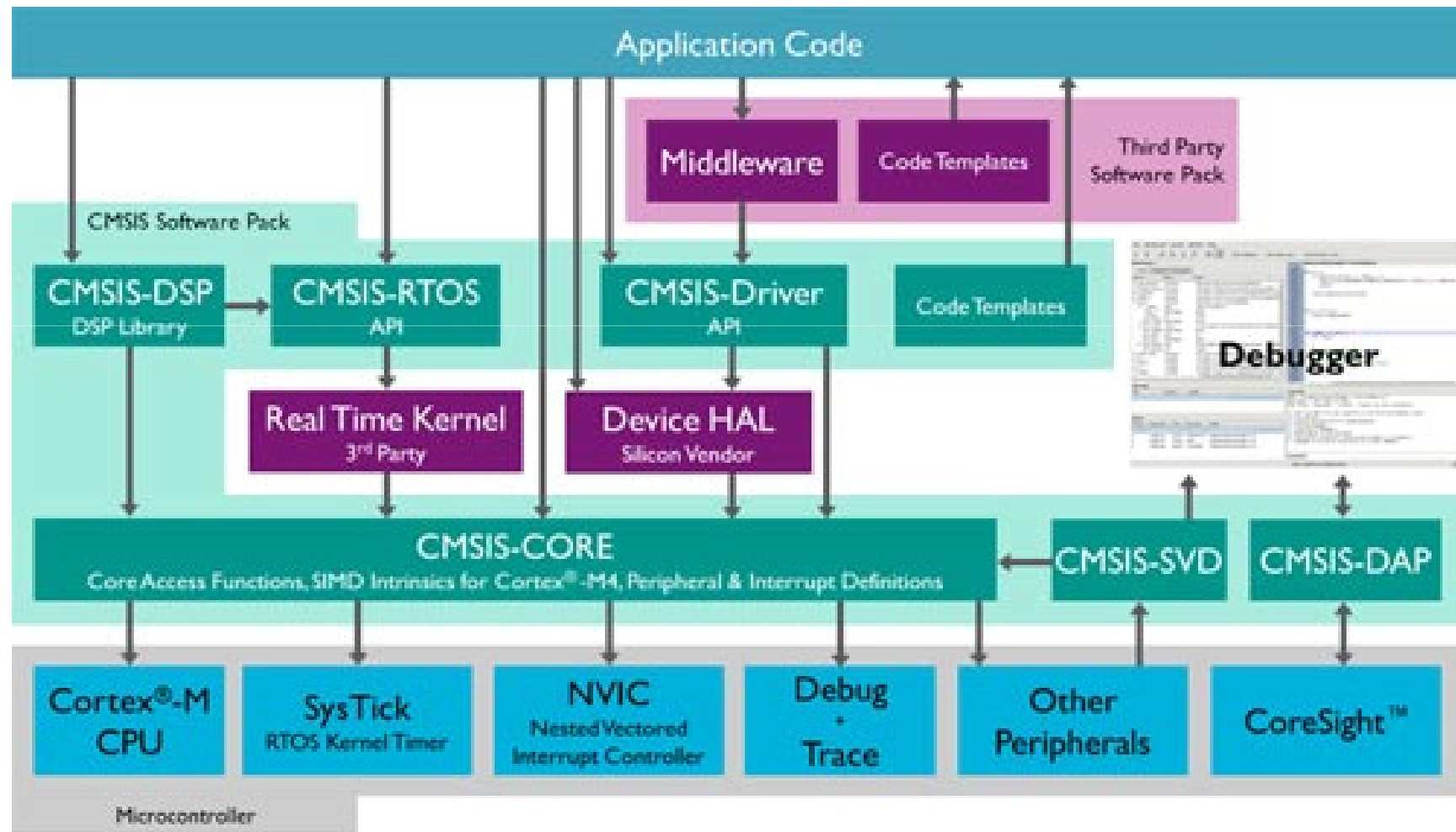
```
GPIO_InitStructure.Mode = GPIO_MODE_OUTPUT_PP;
GPIO_InitStructure.Speed = GPIO_SPEED_HIGH;
GPIO_InitStructure.Pull = GPIO_NOPULL;
HAL_GPIO_Init(GPIOD, &GPIO_InitStructure);
```

```
HAL_GPIO_TogglePin(GPIOD, GPIO_PIN_9); //Toggle the state of pin
```

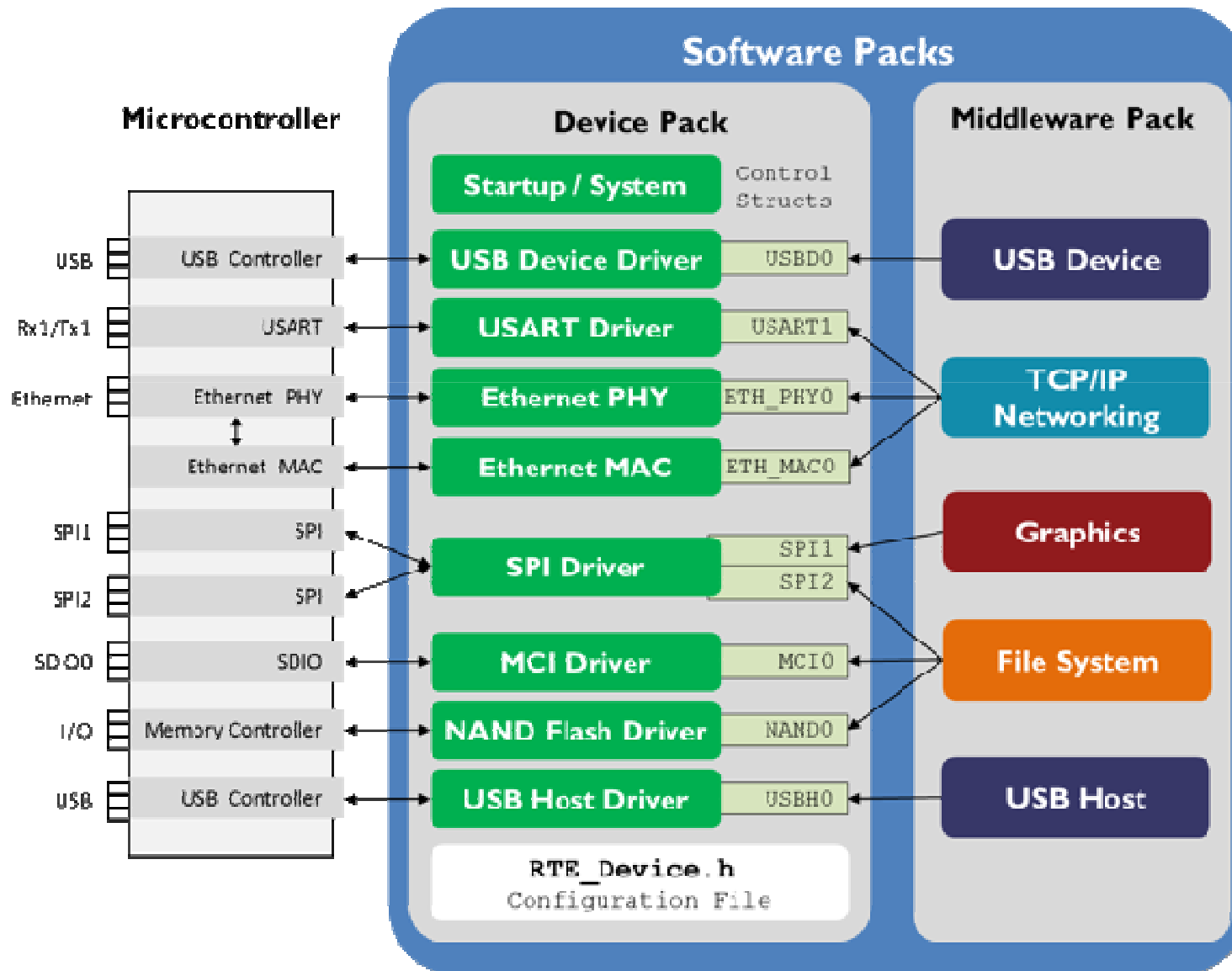
Librairie TM (utilisateur, basé sur HAL)

```
TM_GPIO_Init(GPIOD, GPIO_Pin_12 | GPIO_Pin_13 | GPIO_Pin_14 | GPIO_Pin_15,
             TM_GPIO_Mode_OUT, TM_GPIO_OType_PP, TM_GPIO_PuPd_NOPULL, TM_GPIO_Speed_High);
TM_GPIO_TogglePinValue(GPIOA, GPIO_Pin_5);
```

Structure des libraires



ARM middleware



Conclusions

- Les processeurs ARM sont très prometteurs question performances, flexibilité, fonctionnalité et prix.
- L'augmentation de fonctionnalité résulte en une augmentation de complexité qui est partiellement compensée par la multitude d'exemples
- Il est regrettable que le compilateur Keil soit limité et que le logiciel cube ne soit compatible avec l'IDE Coocox ou semblables.
- ARM sera probablement préférence pour des applications performantes, multi-tâches
- Un environnement style Arduino n'est pas à exclure!
il existe d'ailleurs déjà des circuits style UNO avec ARM

Liens et références

KEIL

<http://www2.keil.com/mdk5>

Coocox

<http://www.coocox.org/software.html>

Source de ma présentation

<http://www.emcu.it/STM32F0xx/STM32F0xx.html>

Divers

<http://www.christoph-lauer.de/archives/11669>

<http://stm32f4-discovery.com>

Fin

Comparaison

Microprocesseurs, Microcontrôleurs

Année/Produits	Transistors	Clock	Mem
74 / Intel 8080	4'500	2-4MHz	64kB adressable
76 / Zilog Z80	8'500	2,5-8	64kB
79 Motorola 68000	68'000	~10MHz	8/16MB 1MIPS
80 Intel 8051	50'000	12MHz	4k 1MIPS
85 Acorn ARM1	25'000	8MHz?	2MB Ram on chip
93 Pentium I	3'300'000	60/66MHz	4GB
00 Pentium 4	42'000'000	>1.3GHz	4GB
11 Quad-Core i7	995'000'000	>2.8GHz	16GB
15 Cortex M4 (ST)	26'000'000	<180MHz	<2MBFlash on chip
15 SPARC M7	10'000'000'000	3.6GHz	1024 cores, 64MB cache

Source: https://en.wikipedia.org/wiki/Transistor_count